リモコン操作ロボット 要件定義

赤外線リモコンで操作する車型ロボット（Arduino Uno）。走行・音楽・センサー

要求1

内容 ユーザーがリモコンのボタン（↑、→、←、↓、OK）を押すと、ロボットが対応する方向に移動または停止する

理由 リモコンでモーター駆動を制御し、ラジコンのように操作するため

すべて開く　すべて閉じる

+ **要求1-1** リモコンの信号をロボットが受信できるようにする
+ **要求1-2** リモコン入力の受付処理を作成する
+ **要求1-3** モーターのピン割り当てを設定する
+ **要求1-4** モーターの移動動作を関数として定義する
+ **要求1-5** リモコンの↑ボタンのコードを取得し、ロボットの前進に対応させる
+ **要求1-6** リモコンの↓ボタンのコードを取得し、ロボットの後退に対応させる
+ **要求1-7** リモコンの→ボタンのコードを取得し、ロボットの右折に対応させる
+ **要求1-8** リモコンの←ボタンのコードを取得し、ロボットの左折に対応させる
+ **要求1-9** リモコンのOKボタンのコードを取得し、ロボットの停止に対応させる
+ **要求1-10** 電源を入れた直後のロボットの状態（初期値）を決める
+ **要求1-11** OKボタンが押されたら、ロボットを完全に停止させる

+ **要求1-11** OKボタンが押されたら、ロボットを完全に停止させる

+ **要求1-12** 右折・左折のときのタイヤの動きを決める

+ **要求1-13** ボタンの同時押しや、定義されていない信号を受けたときの動作を決める

要求2

内容 リモコンの数字ボタン（1〜9）を押すと、対応する曲がブザーから流れる

理由 走行だけでなく、音楽でも楽しめるようにするため

+ **要求2-1** ブザーを接続する

+ **要求2-2** 数字ボタンのコードを取得する

+ **要求2-3** 音階と曲データを用意する

+ **要求2-4** 音を鳴らす関数 `play_Tone()` を作成する

+ **要求2-5** 曲を再生する関数 `play_Music(番号)` を作成する

+ **要求2-6** 数字ボタンと曲を対応させる

要求3

内容 センサーで障害物を見分け、優先度に従ってロボットの動きを決める

理由 ぶつからないようにし、リモコンの指示よりも安全を優先するため

+ **要求3-1** 処理を「入力→判断→出力」の3段階に分ける

+ **要求3-2** 超音波センサーで前方の障害物を検知する

+ **要求3-3 赤外線センサーで後方の障害物を検知する**

+ **要求3-4 センサーの結果から、障害物の状態を判断する**

要求4

内容 障害物の状態ごとに、ロボットの動きと優先度を決める

理由 障害物があるときは、ユーザーの指示より安全を優先して自動で避けるため

+ **要求4-1 前方に障害物があるとき（超音波センサーが検知）**

+ **要求4-2 後方に障害物があるとき（赤外線センサーが検知）**

+ **要求4-3 前後の両方に障害物があるとき（両方のセンサーが検知）**

+ **要求4-4 障害物がないとき（リモコン入力）**

+ **要求4-5 優先度のイメージ（前進ボタンを押して前方に壁があるとき）**

− **要求4-6 回避動作の終わり方を決める**

理由 前後の壁の間を行ったり来たりし続けるのを防ぐため

4-6-1	回避動作は一定時間（ `AVOID_MS` ＝0.5秒）だけ行い、その後 `GO_STOP` で停止する
4-6-2	回避動作中は、OKボタン以外のボタン（走行・数字）を受け付けない
4-6-3	回避動作中に、後退して後方を検知したとき、または前進して前方を検知したときは、逆向きに切り替えず停止する
4-6-4	停止後は、ユーザーが次のボタンを押すまで待つ

※元の資料に書かれていなかったため、追加した決まりです。

要求5

内容 ブザーで、警告音と音楽を鳴らす

理由 障害物を避けたことを音で知らせ、曲も楽しめるようにするため

+ **要求5-1 ブザーの共通仕様を決める**

+ **要求5-2 警告音 `warning()` を作成する**

+ **要求5-3 音の優先順位を決める**

要求6

内容 複数の条件が重なったときの優先順位と、処理の順番を決める

理由 状況が重なっても動作が1つに決まり、安全側に動くようにするため

+ **要求6-1 動作の優先順位を決める**

+ **要求6-2 メインループの処理順を決める**

要求7

内容 要求どおりに動くかを確認する

理由 組み立て後に、安全に動くことを確かめるため

+ **要求7-1 走行と停止を確認する**

+ **要求7-2 センサーによる回避を確認する**

+ **要求7-3 音楽を確認する**

図

シーケンス図

図1　起動（電源を入れたとき）

setup が各部を初期化します。モーターは停止、ブザーは停止のまま、ボタンを待ちます。

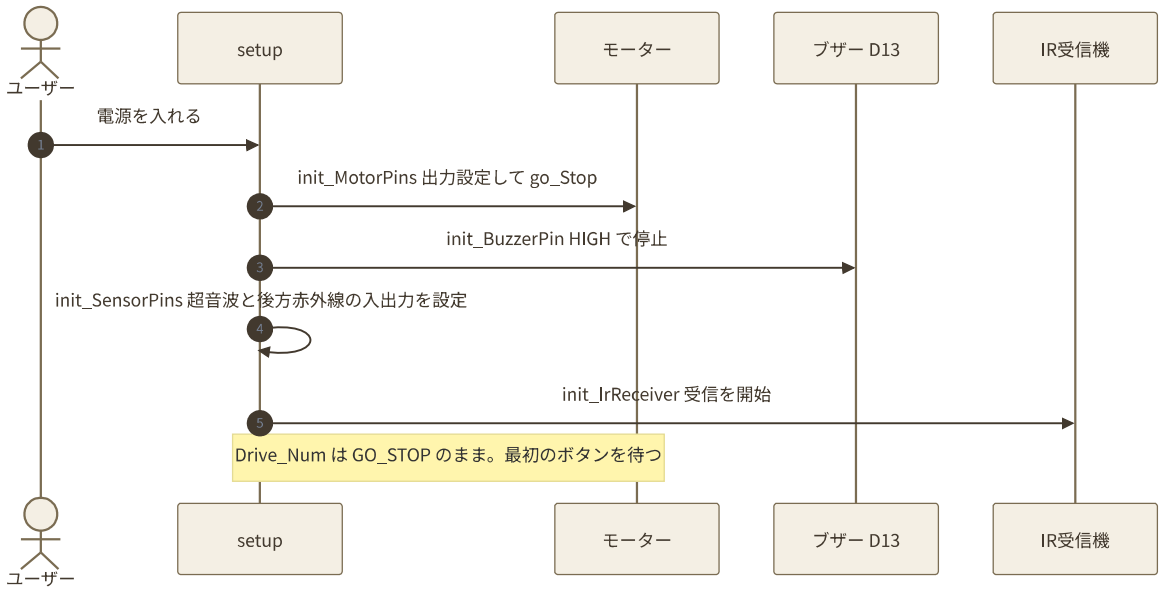


図2　前進ボタンと前方の壁（OKで停止まで）

入力→判断→出力の順に進み、壁があると判断が状態を後退に上書きします。最後にOKで停止します。

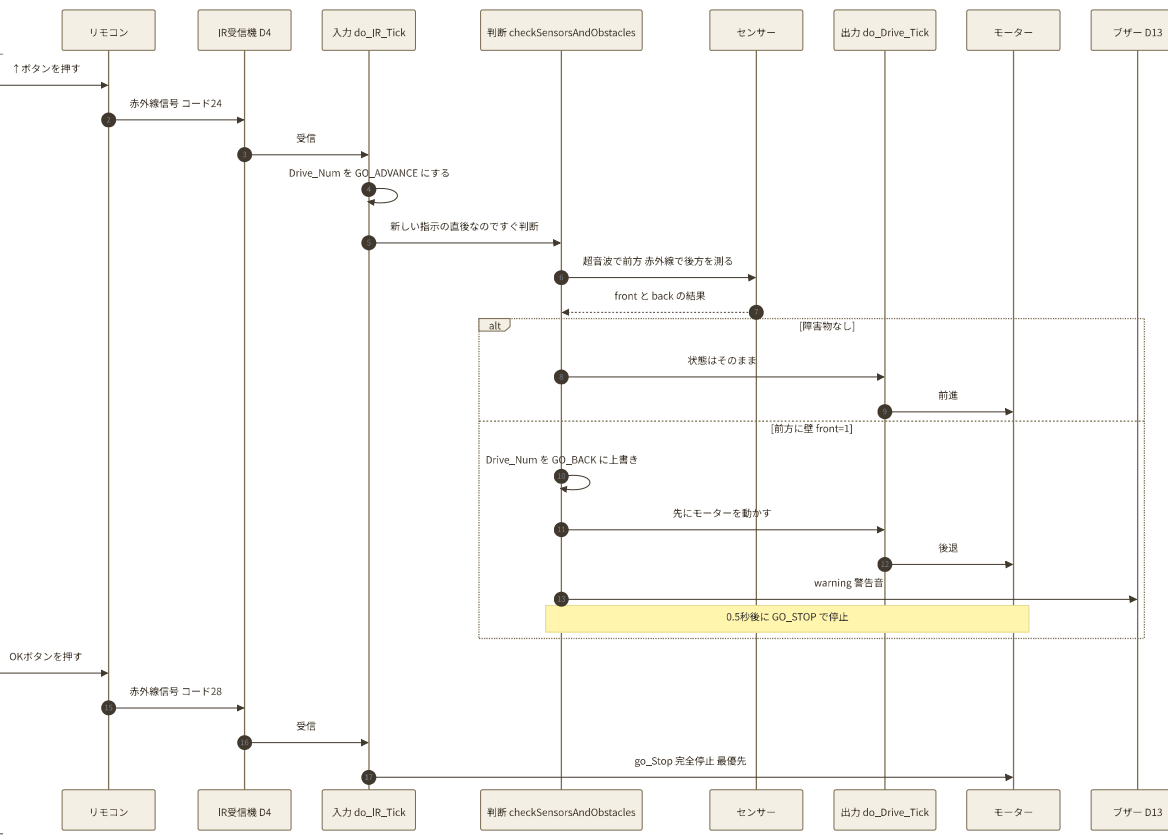


図3　後退ボタンと後方の壁／前後の両方に壁

後方だけなら前進、前後の両方なら回転に切り替え、どちらも警告音を鳴らします。

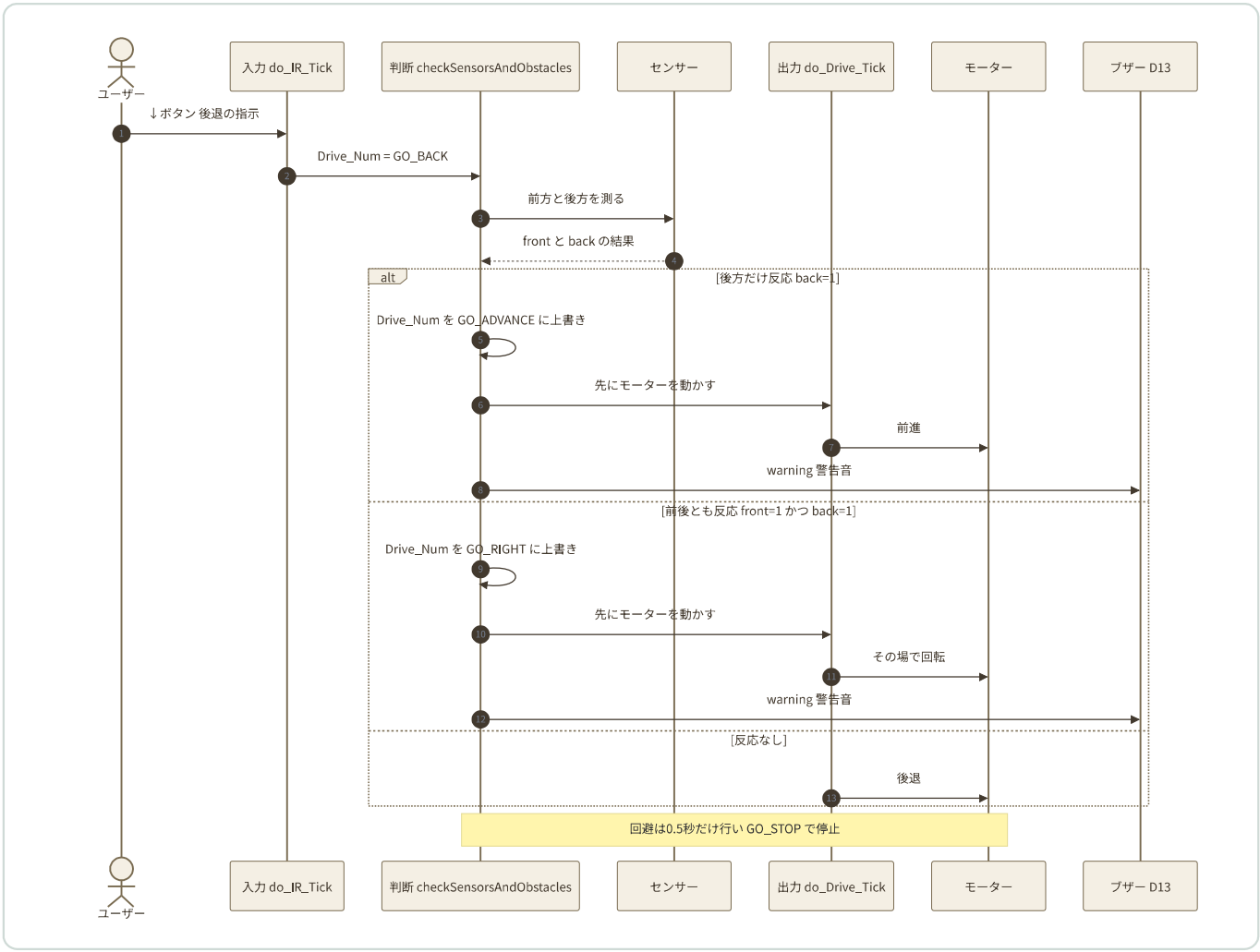
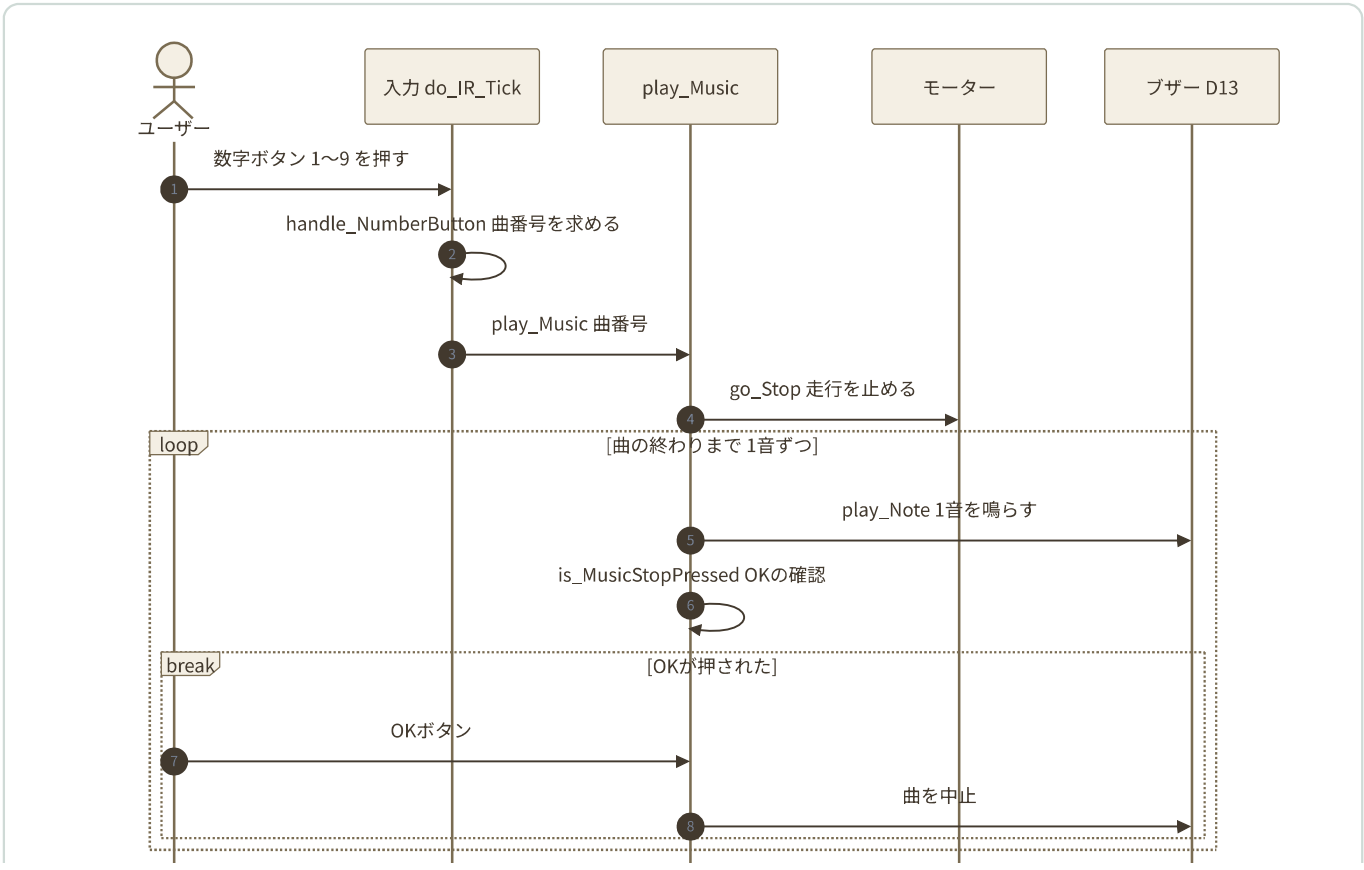


図4　数字ボタンで音楽を再生（OKで中止）

再生の前にモーターを止めます。1音ごとにOKが押されたか確認し、押されたら曲を中止します。



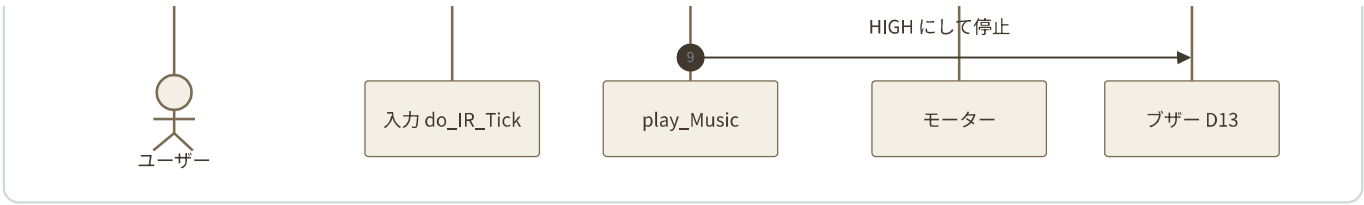
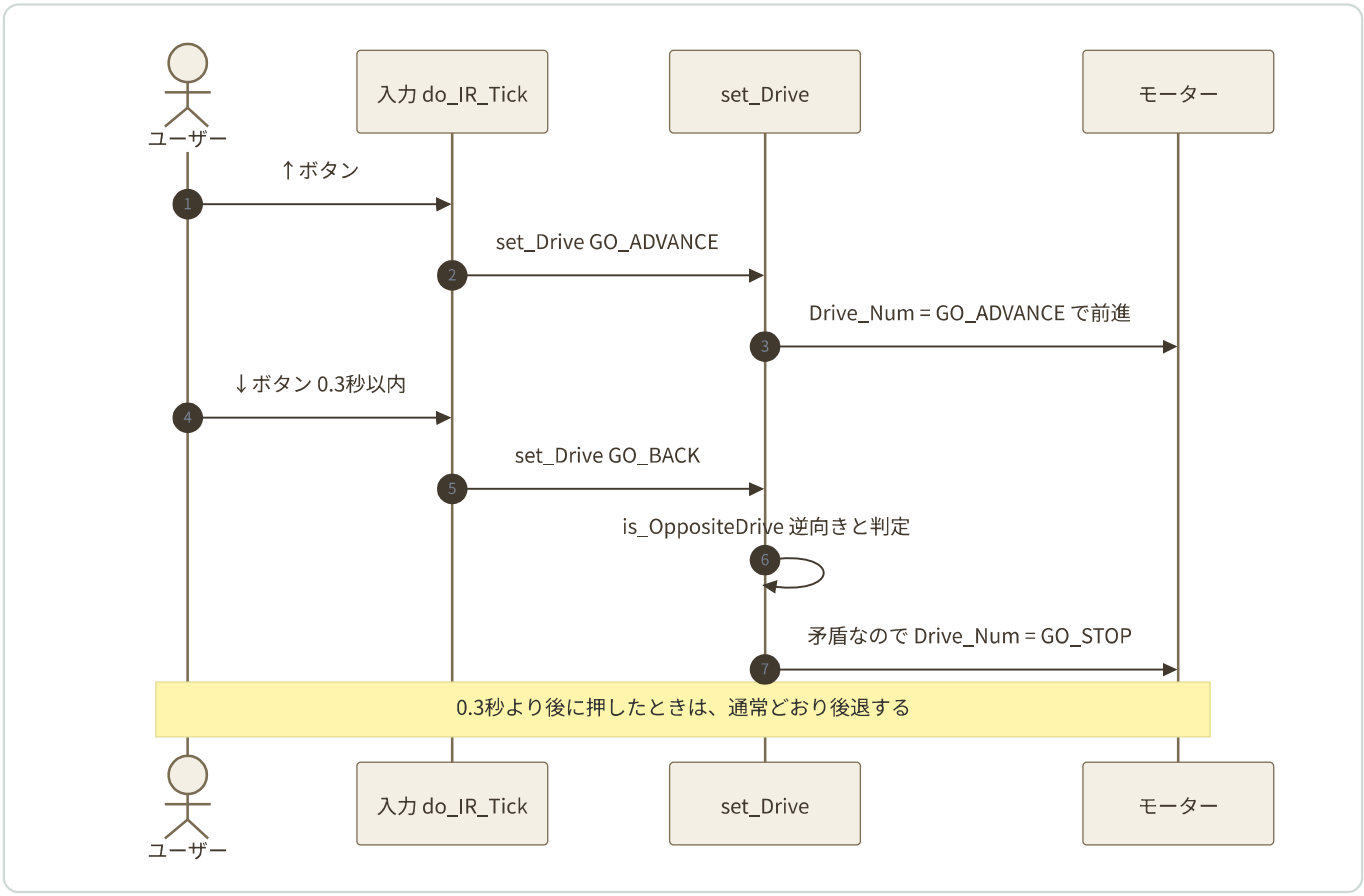


図5　逆向きのボタンを続けて押したとき

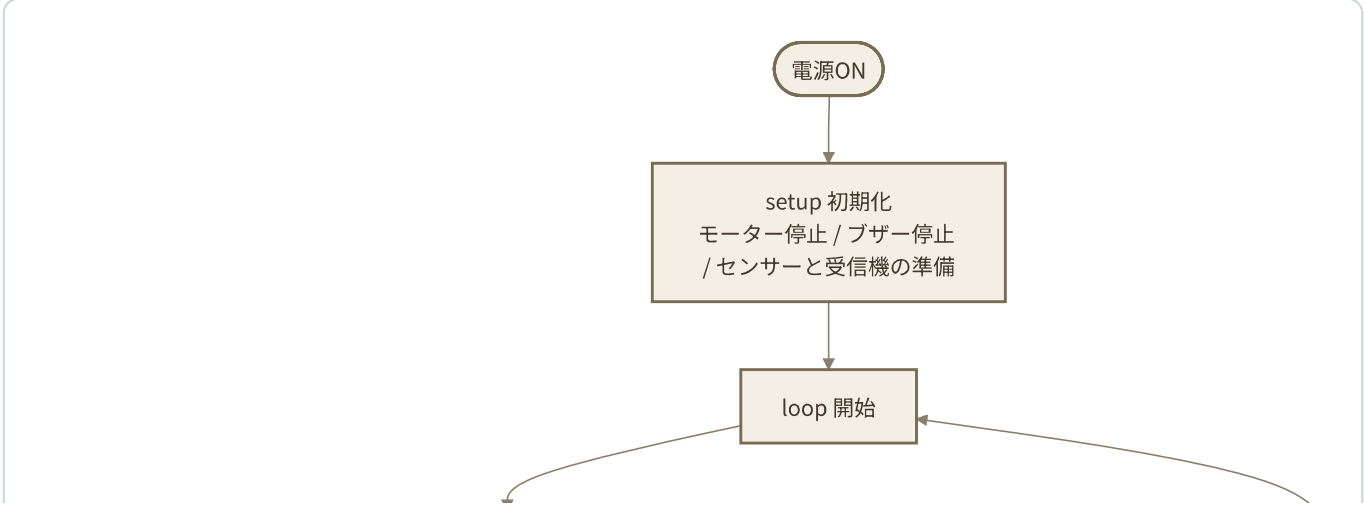
↑と↓、または←と→を0.3秒以内に続けて押すと、矛盾とみなして停止します。

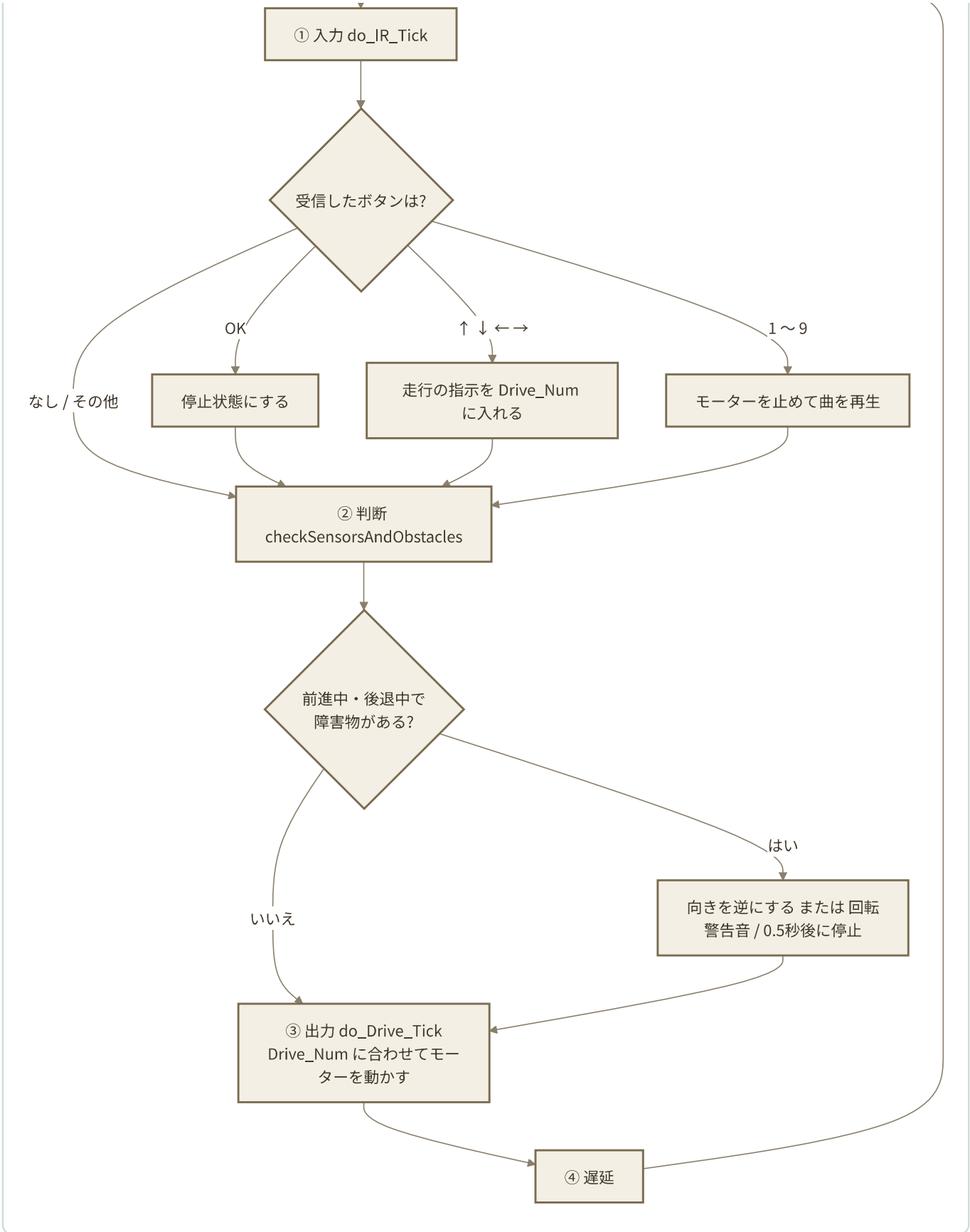


フローチャート：全体の流れ

図6　全体の流れ

電源ONから、入力・判断・出力を繰り返す流れです。各処理の中身は、次の関数別フローチャートで説明します。





フローチャート：関数別

図7　関数 setup

初期化の関数を順に呼びます。

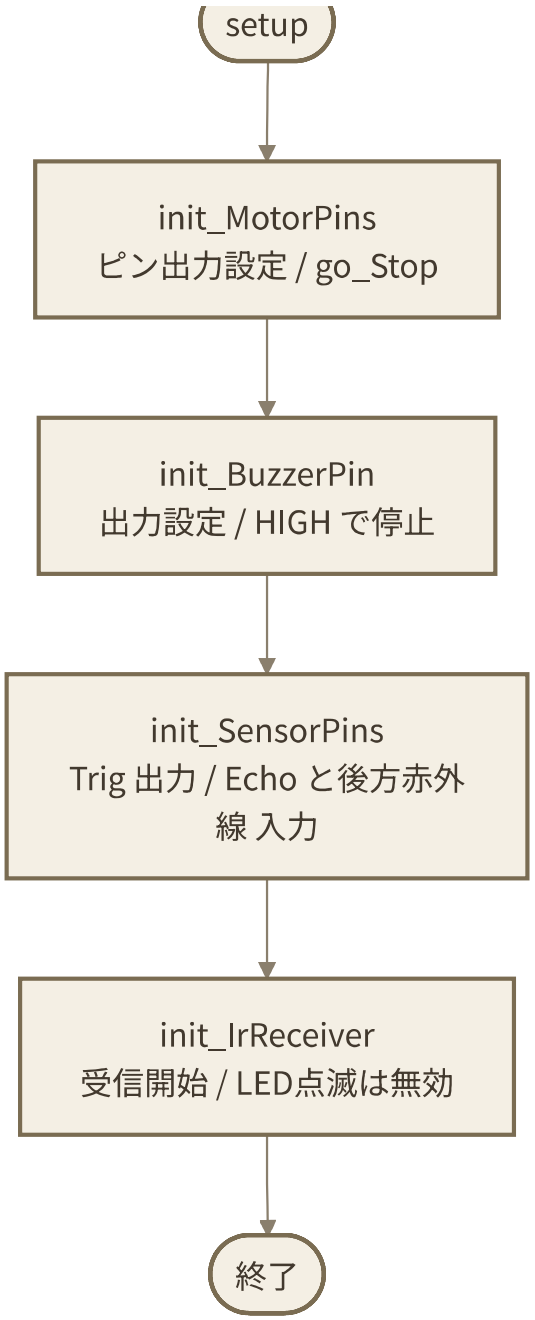
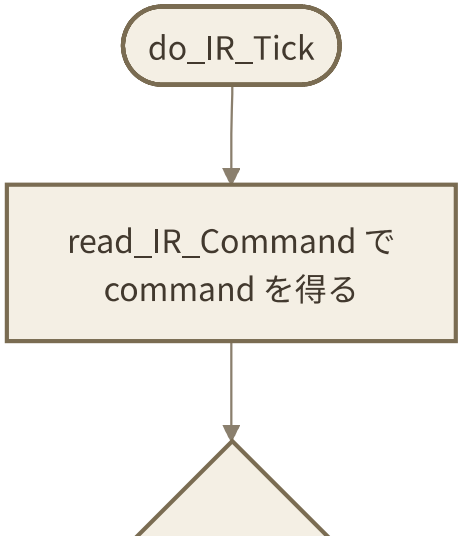


図8　関数 do_IR_Tick

受信した1件を、ボタンの種類ごとの関数へ振り分けます。



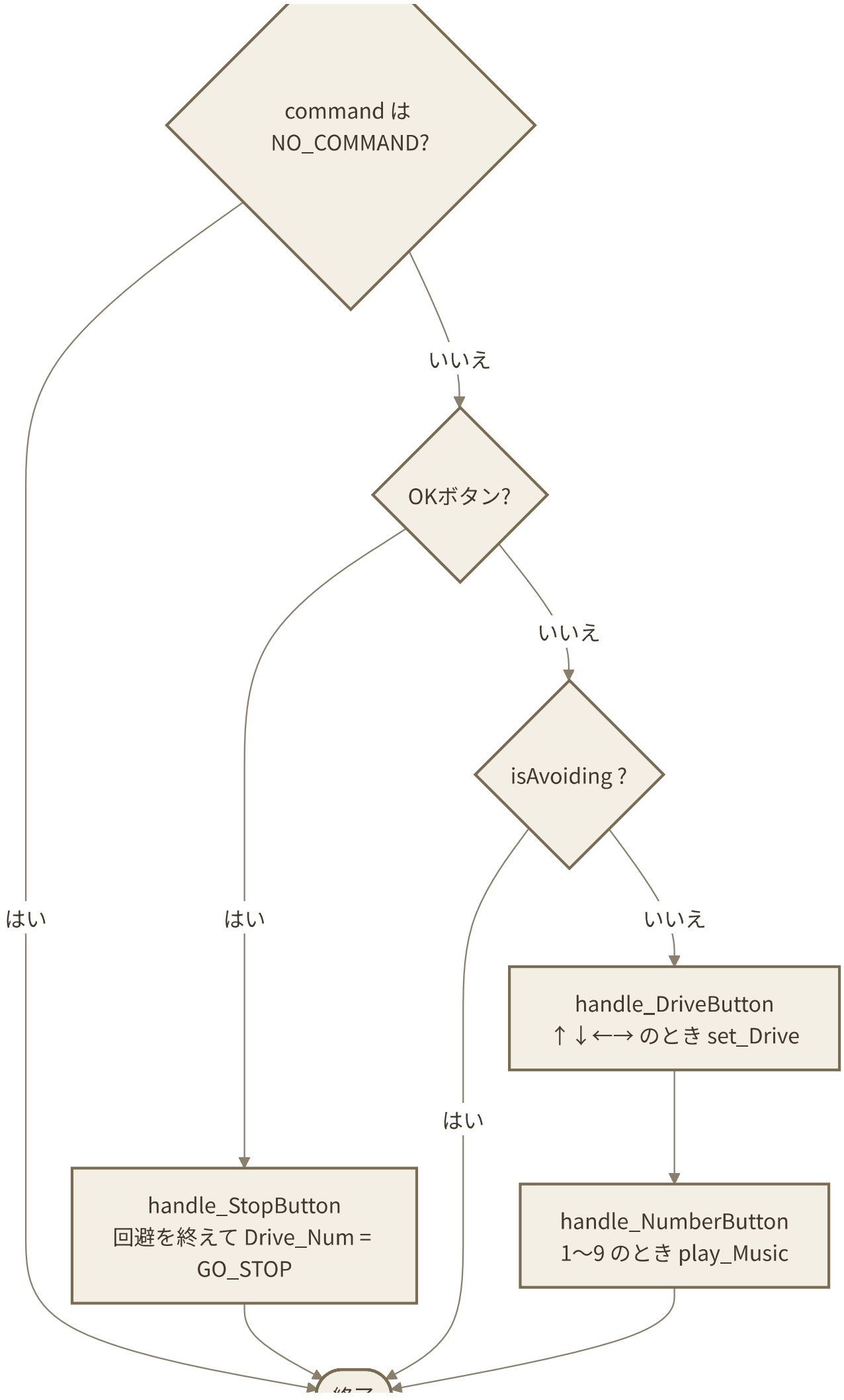

command は NO_COMMAND?
いいえ
OKボタン?
いいえ
isAvoiding ?
はい
はい
はい
いいえ
handle_DriveButton
↑↓←→ のとき set_Drive
handle_StopButton
回避を終えて Drive_Num = GO_STOP
handle_NumberButton
1〜9 のとき play_Music



図9　関数 read_IR_Command

受信コードを返します。受信なしとリピート信号は NO_COMMAND です。

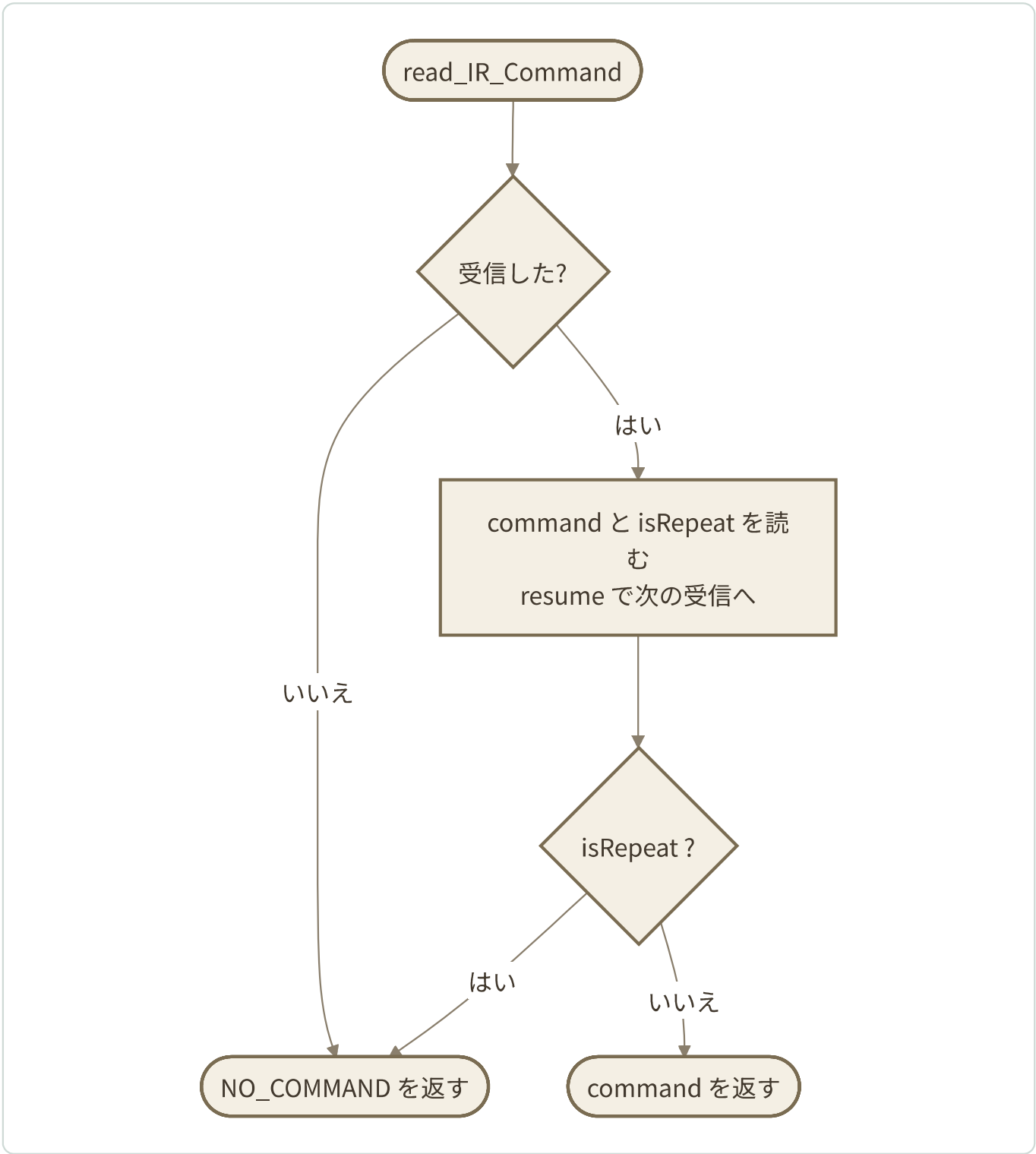
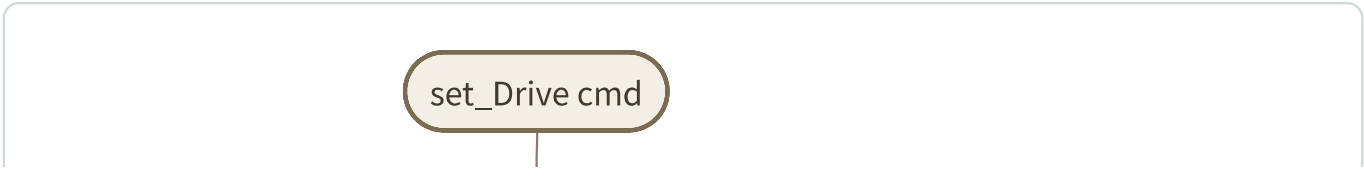


図10　関数 set_Drive

走行の指示を受け付けます。逆向きの指示が続いたら停止します。



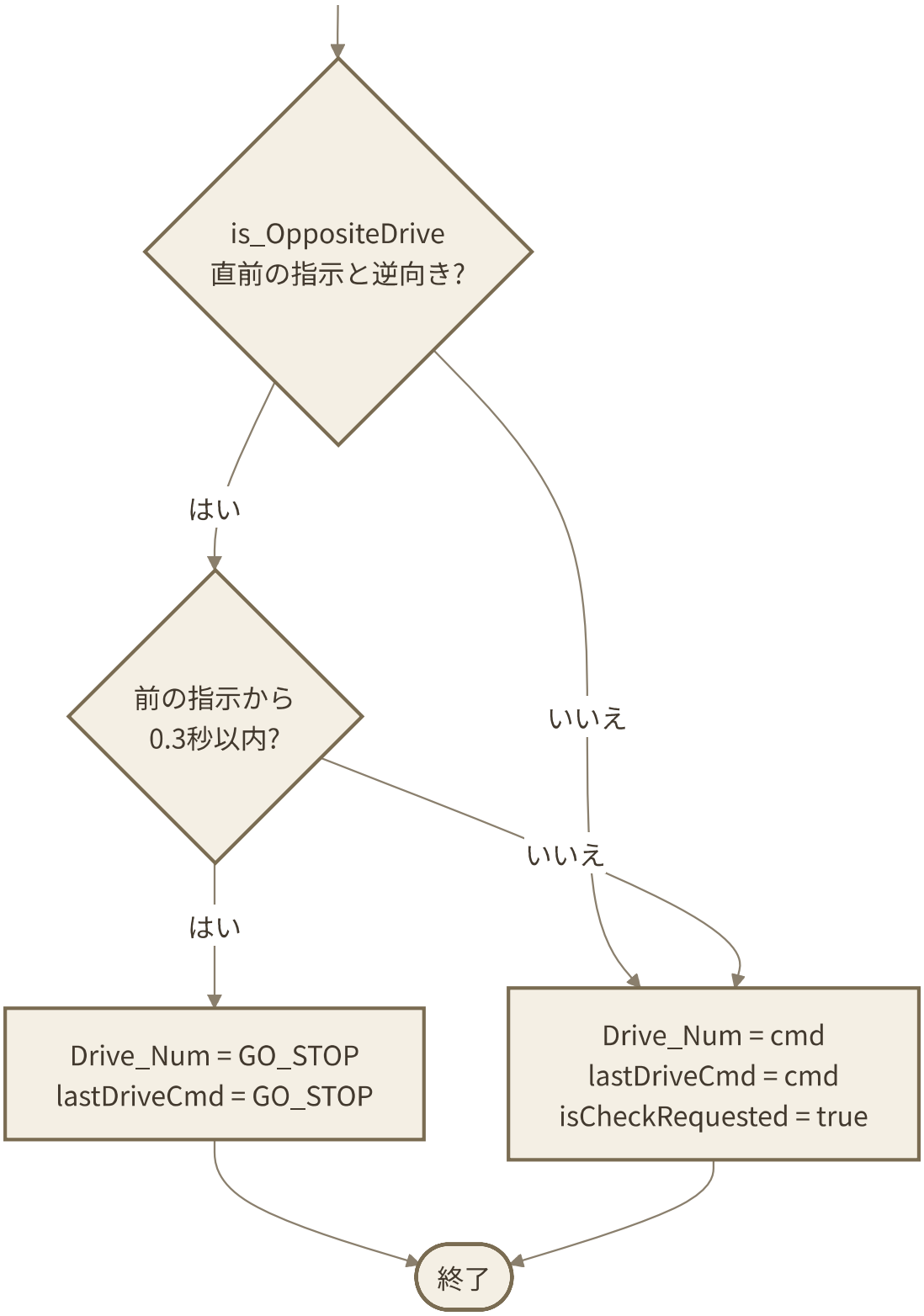
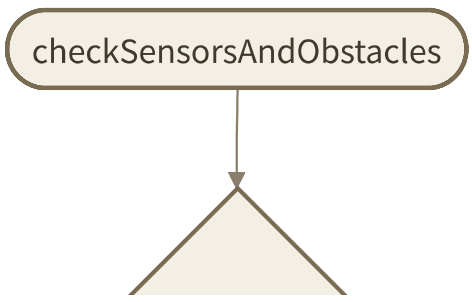


図11　関数 checkSensorsAndObstacles

判断の手順を順に呼ぶだけの関数です。



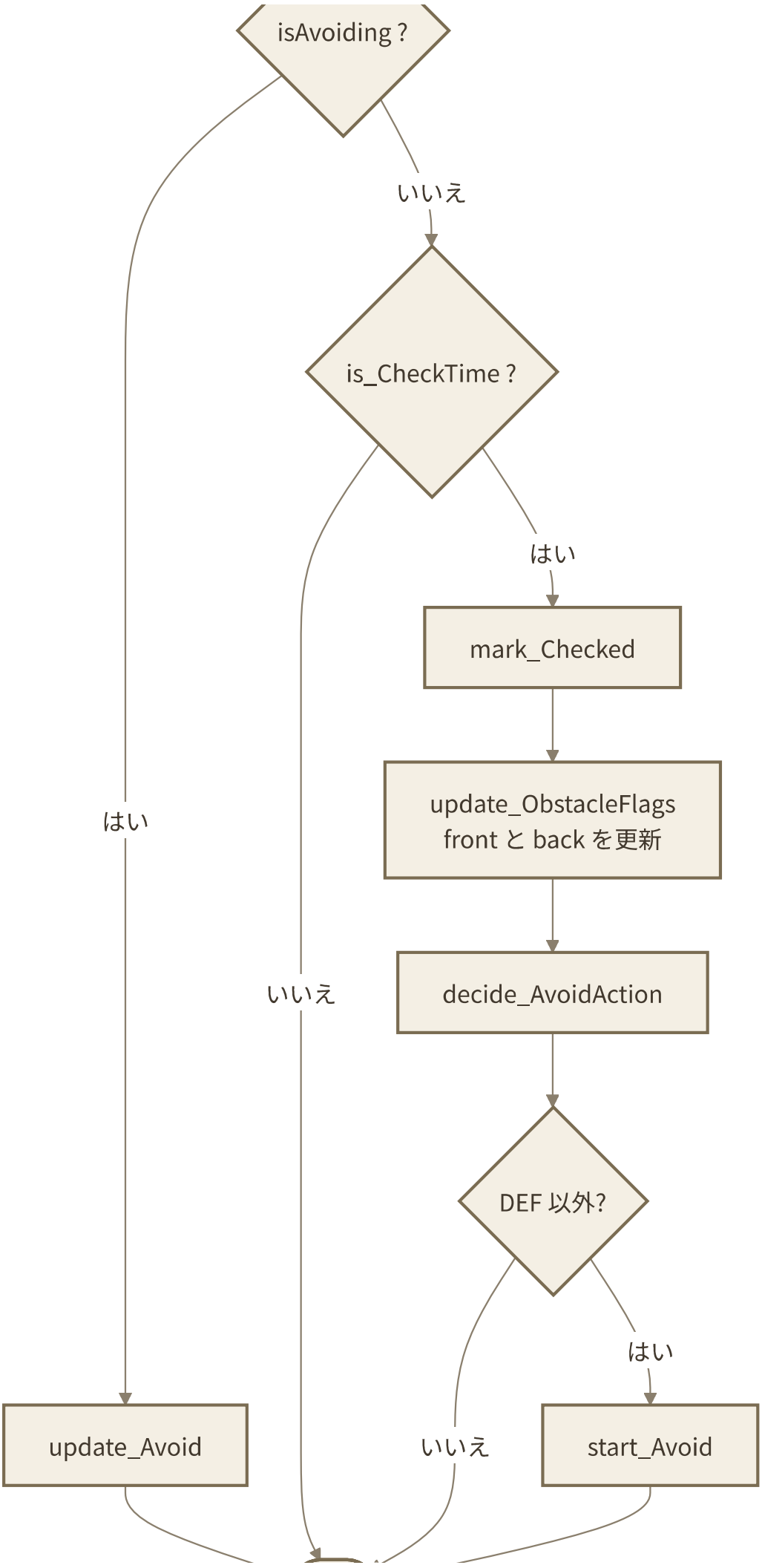

isAvoiding ?
いいえ
is_CheckTime ?
はい
mark_Checked
update_ObstacleFlags
front と back を更新
decide_AvoidAction
DEF 以外?
はい
start_Avoid
いいえ
はい
update_Avoid
いいえ



図12　関数 update_Avoid

回避動作中に、終わらせるかどうかを調べます。

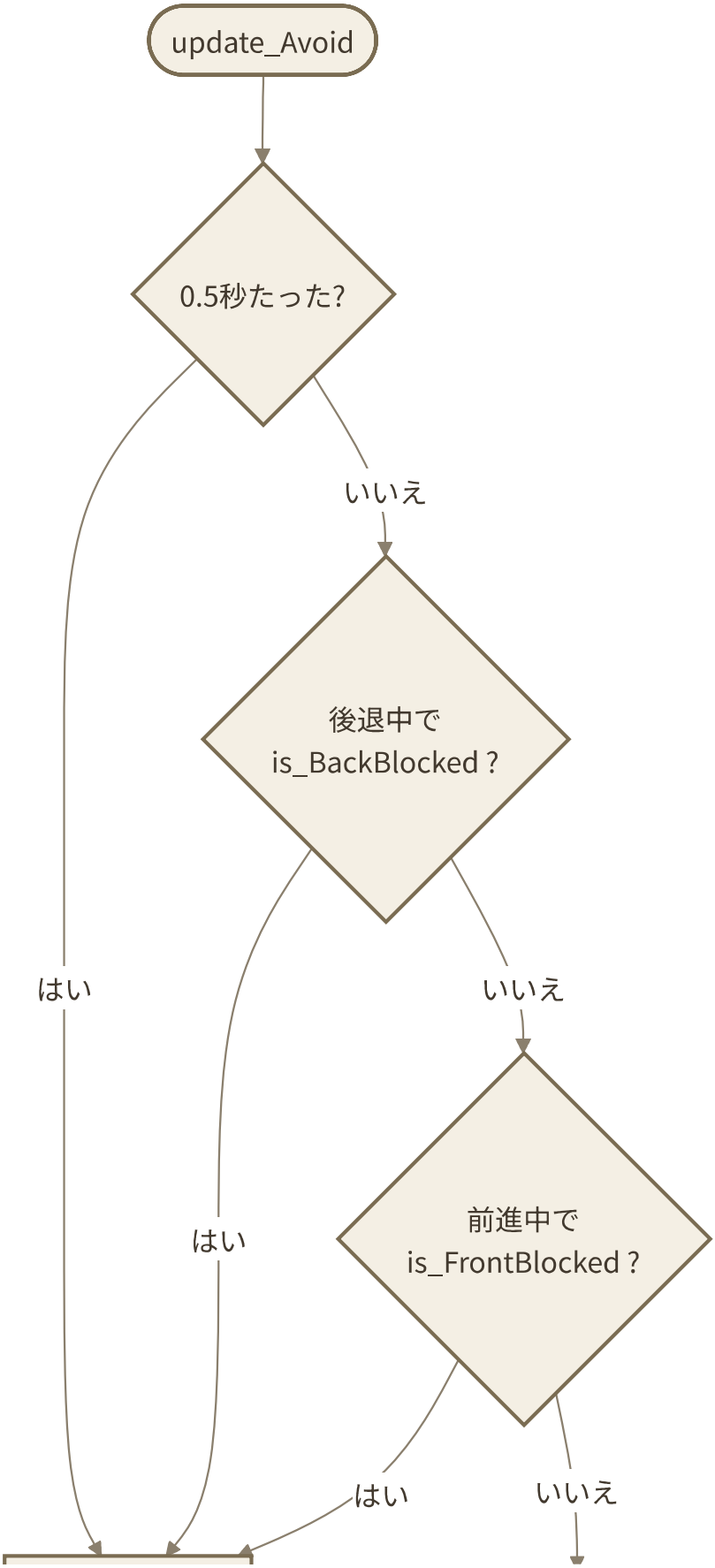




図13　関数 is_CheckTime

いま障害物を調べるかどうかを返します。

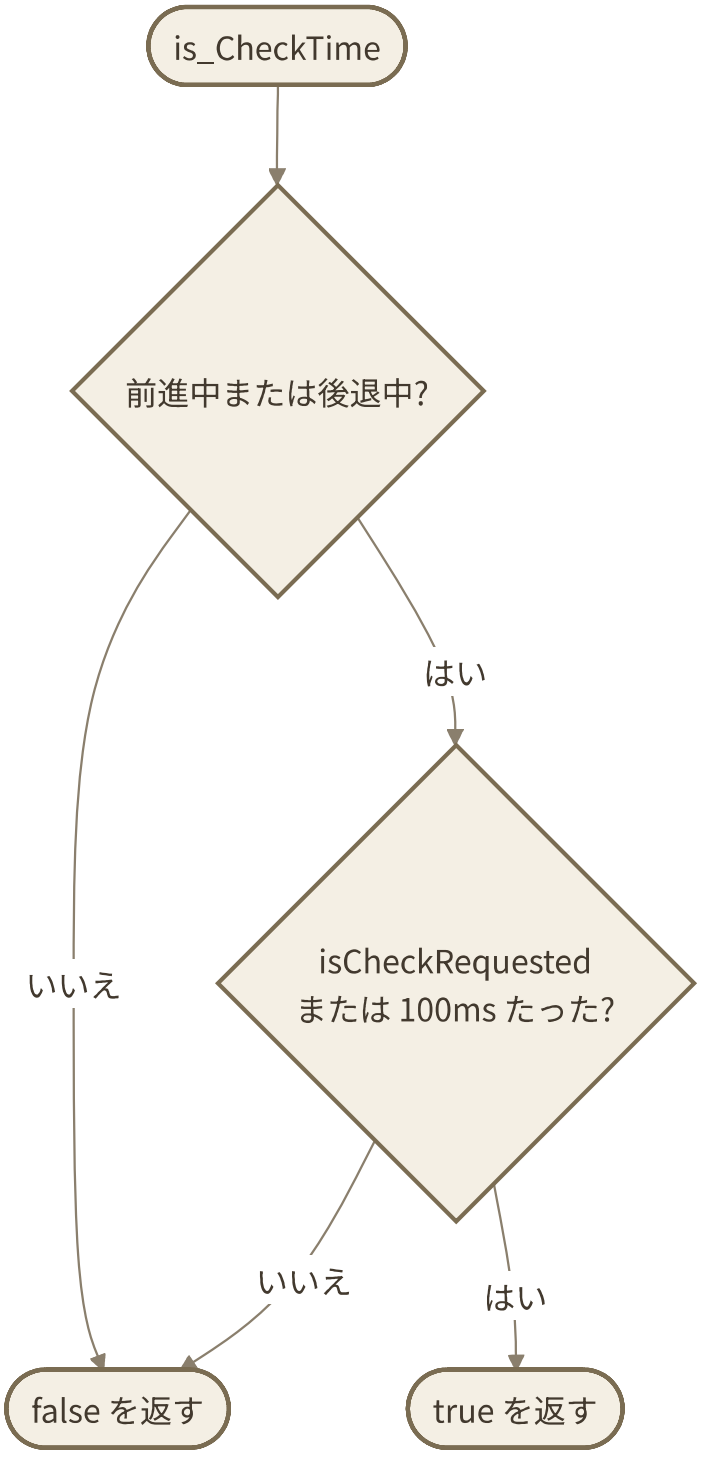


図14　関数 decide_AvoidAction

図14　関数 decide_AvoidAction

front・back・Drive_Num から、回避動作を返します。

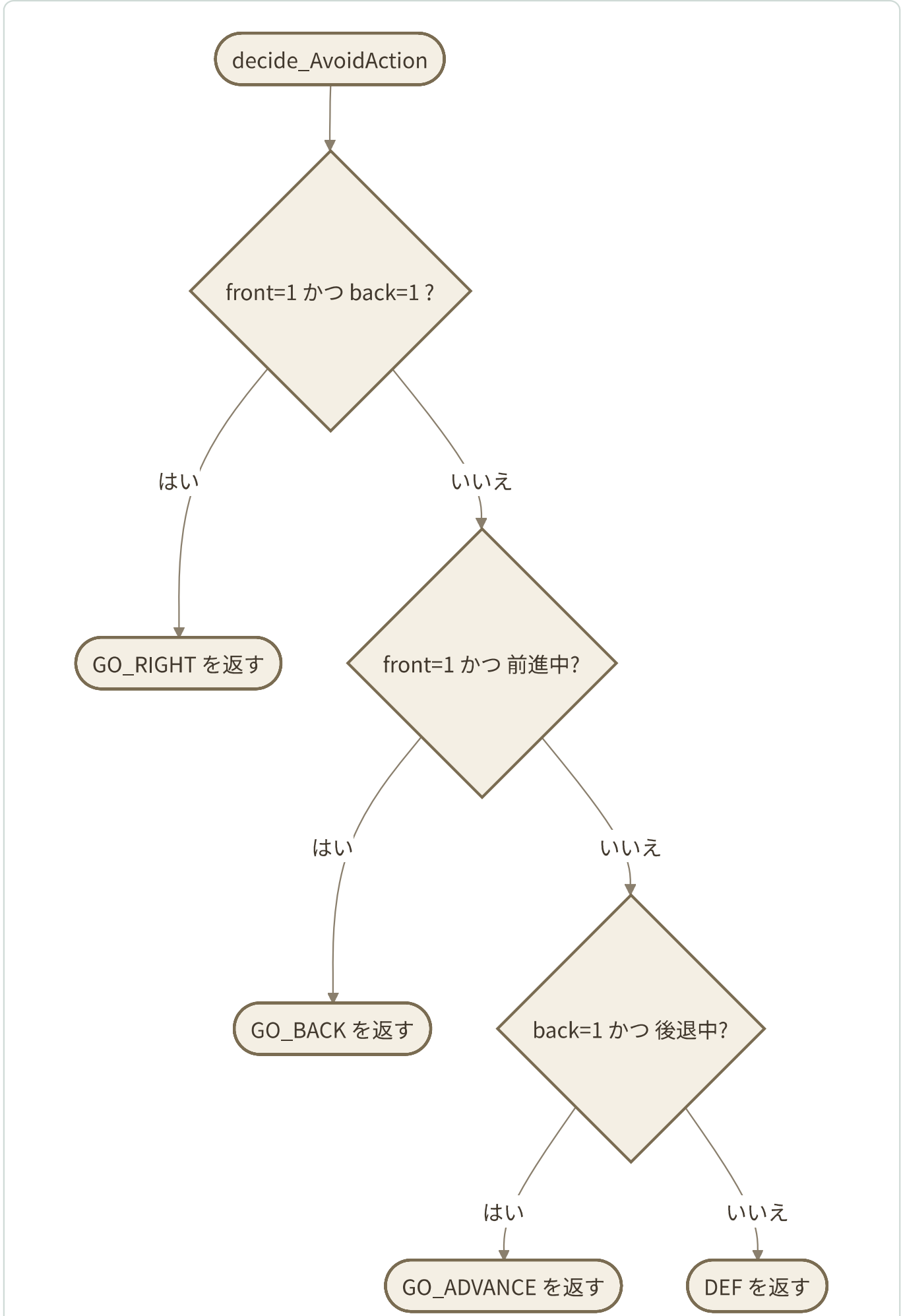


図15　関数 start_Avoid と end_Avoid

回避を始める関数と、終える関数です。



図16　関数 is_FrontBlocked と is_BackBlocked

どちらも、2回続けて検知したときだけ true を返します。

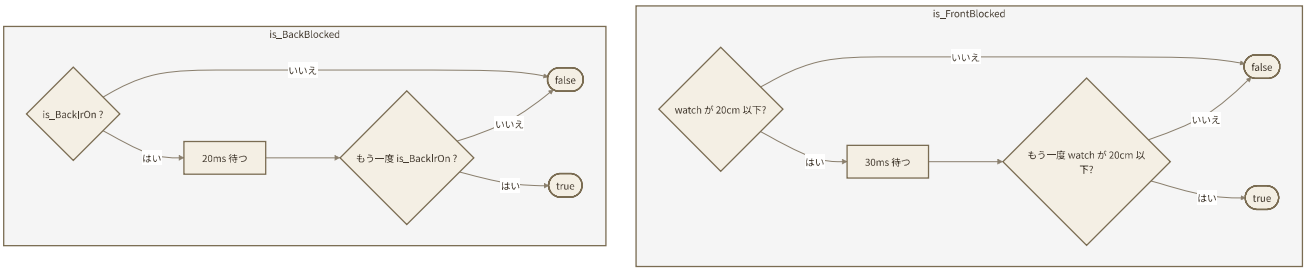
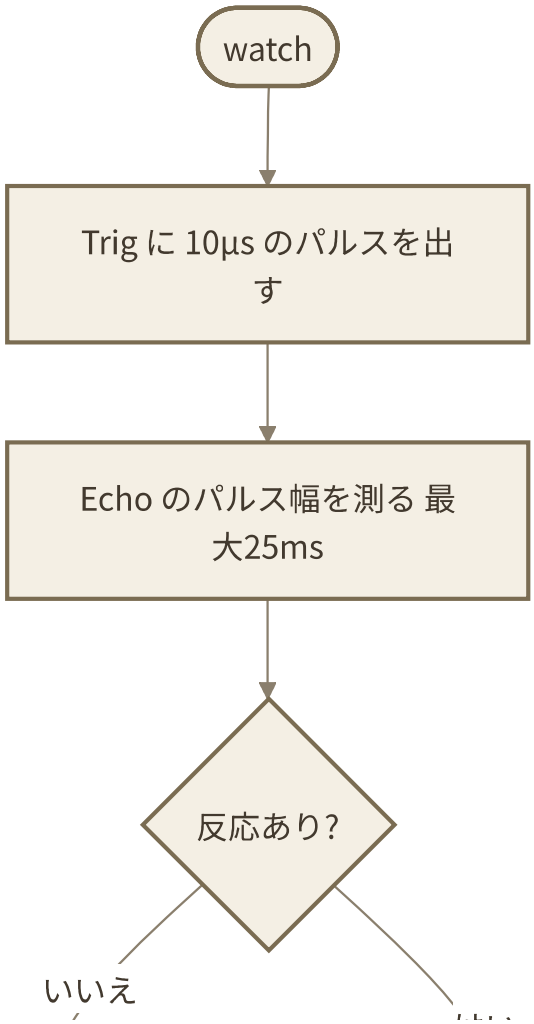


図17　関数 watch

超音波センサーで前方の距離（cm）を返します。



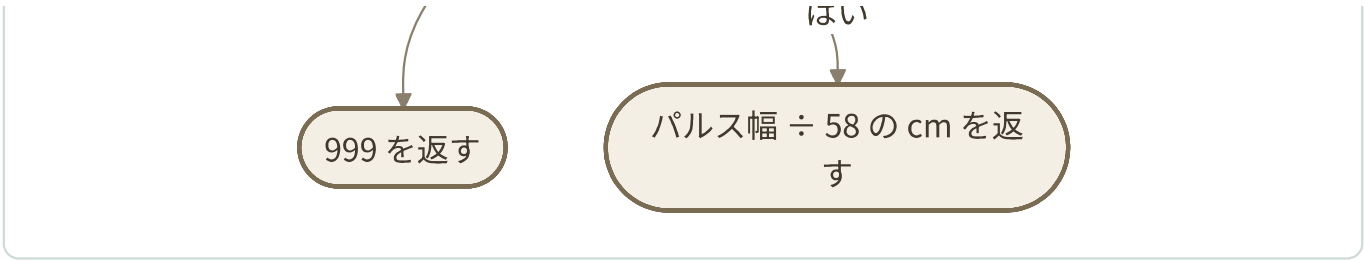


図18　関数 do_Drive_Tick

Drive_Num に合う動作関数を呼びます。

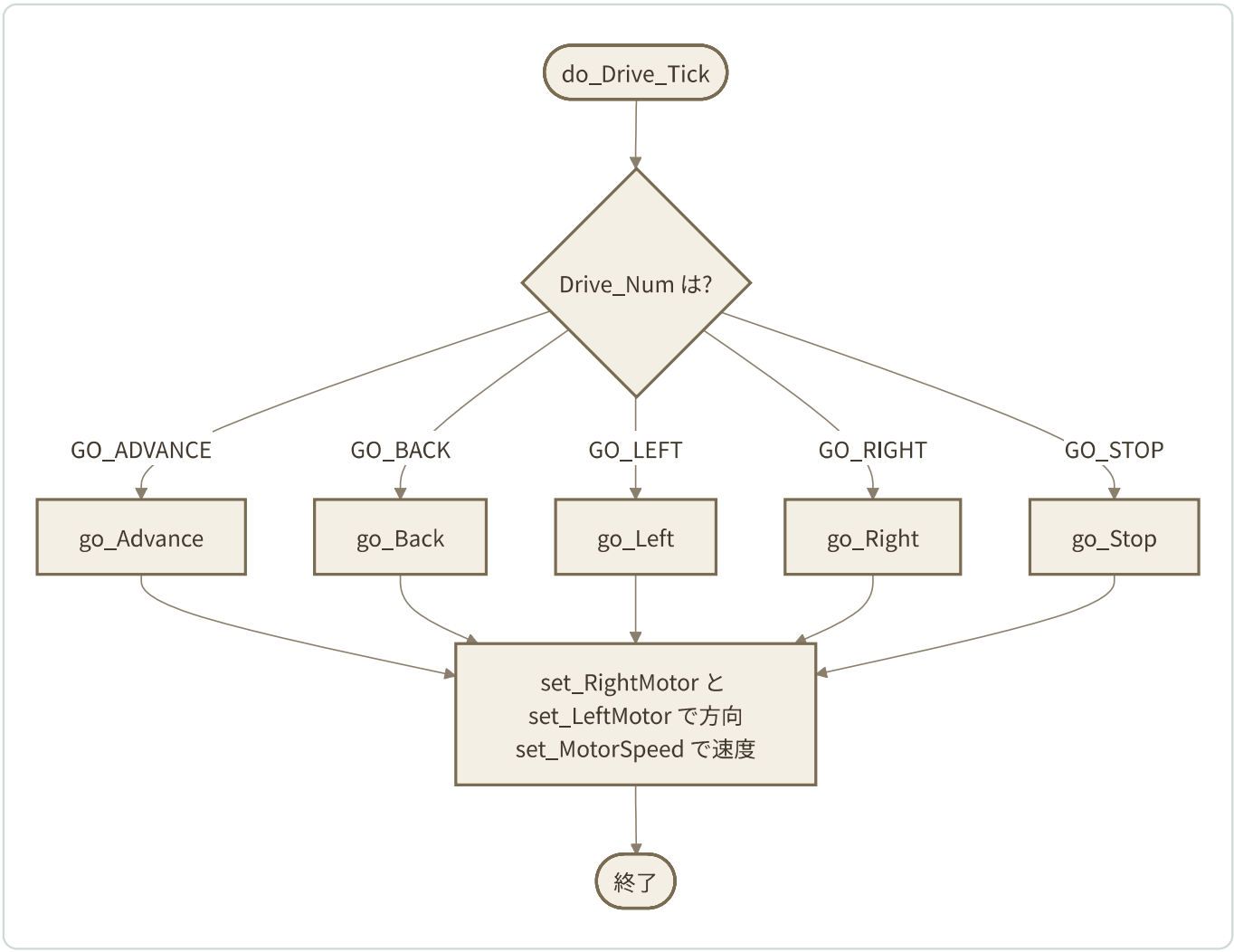
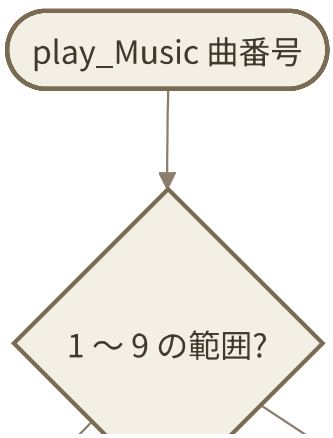
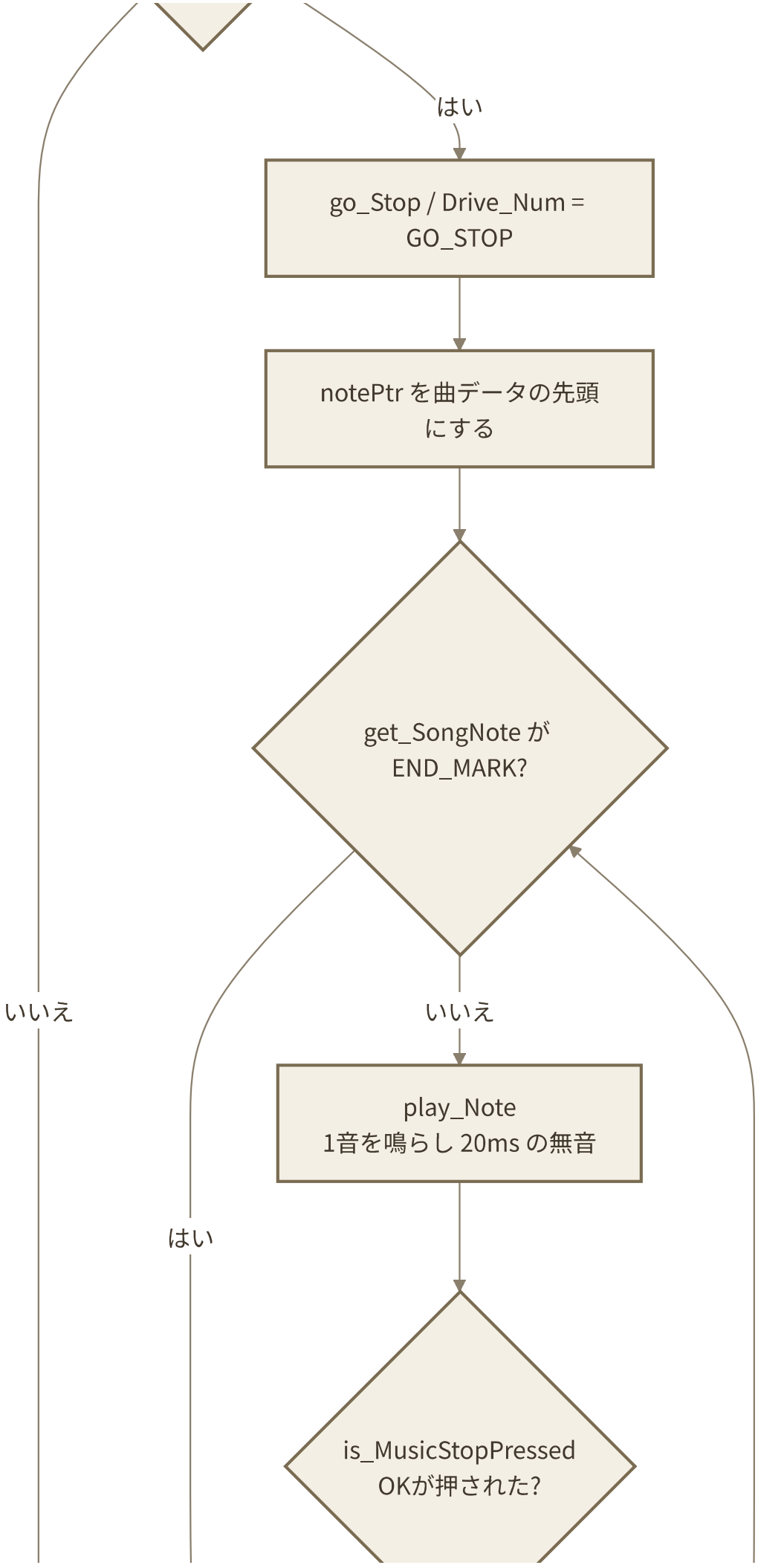


図19　関数 play_Music

曲番号の曲を、最後まで（またはOKが押されるまで）再生します。




はい
go_Stop / Drive_Num =
GO_STOP
notePtr を曲データの先頭
にする
get_SongNote が
END_MARK?
いいえ
いいえ
play_Note
1音を鳴らし 20ms の無音
はい
is_MusicStopPressed
OKが押された?

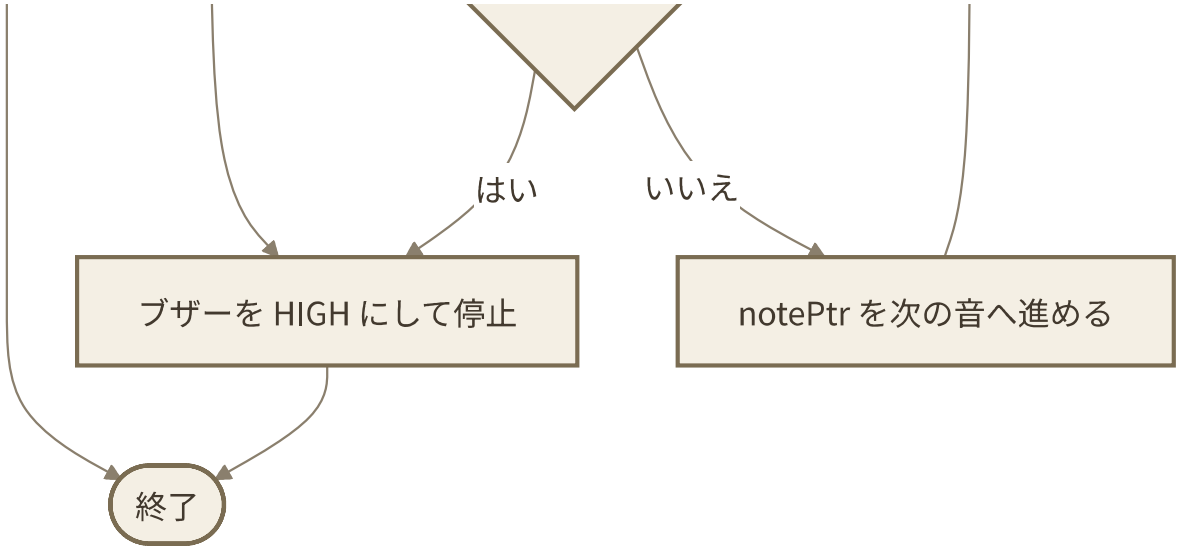


図20　関数 warning

警告音を鳴らします。1秒に1回までです。

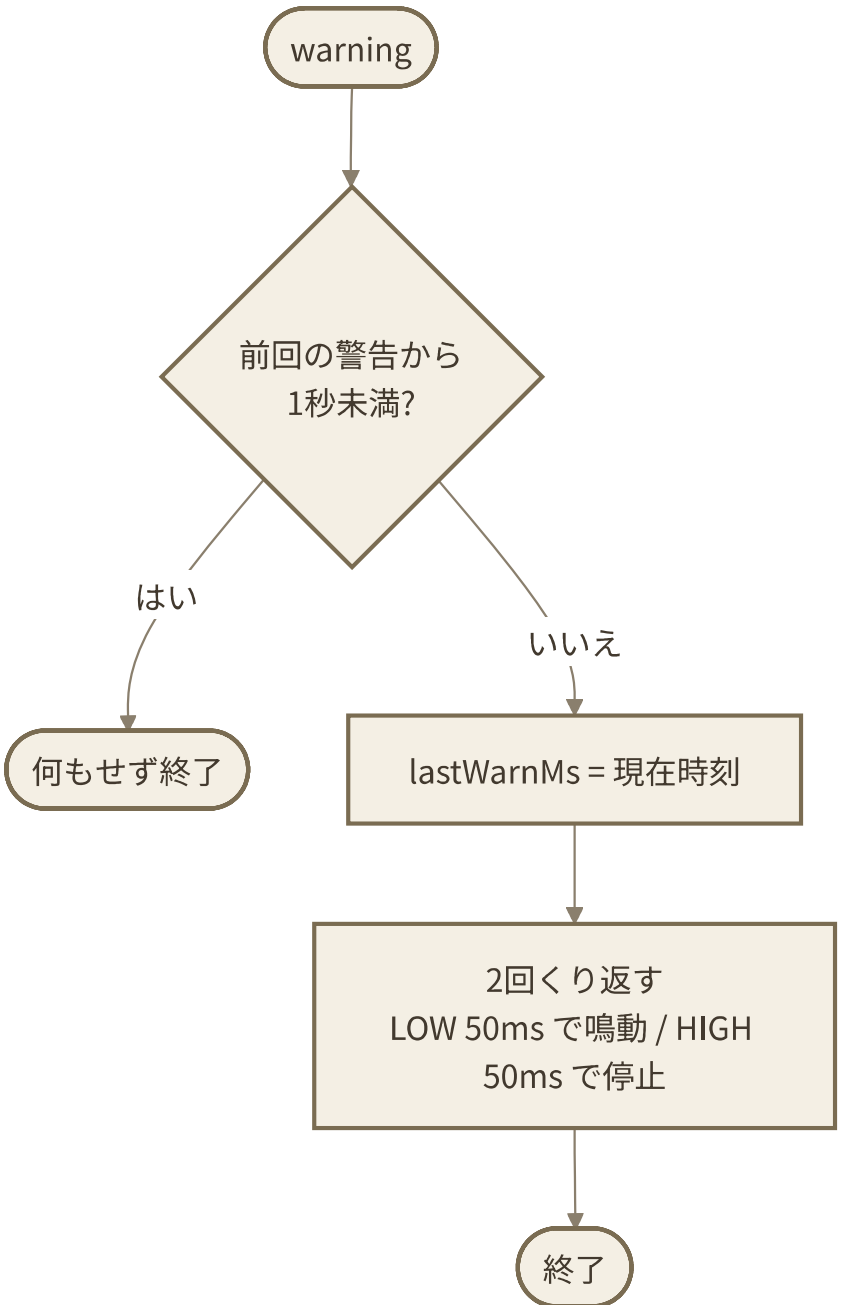
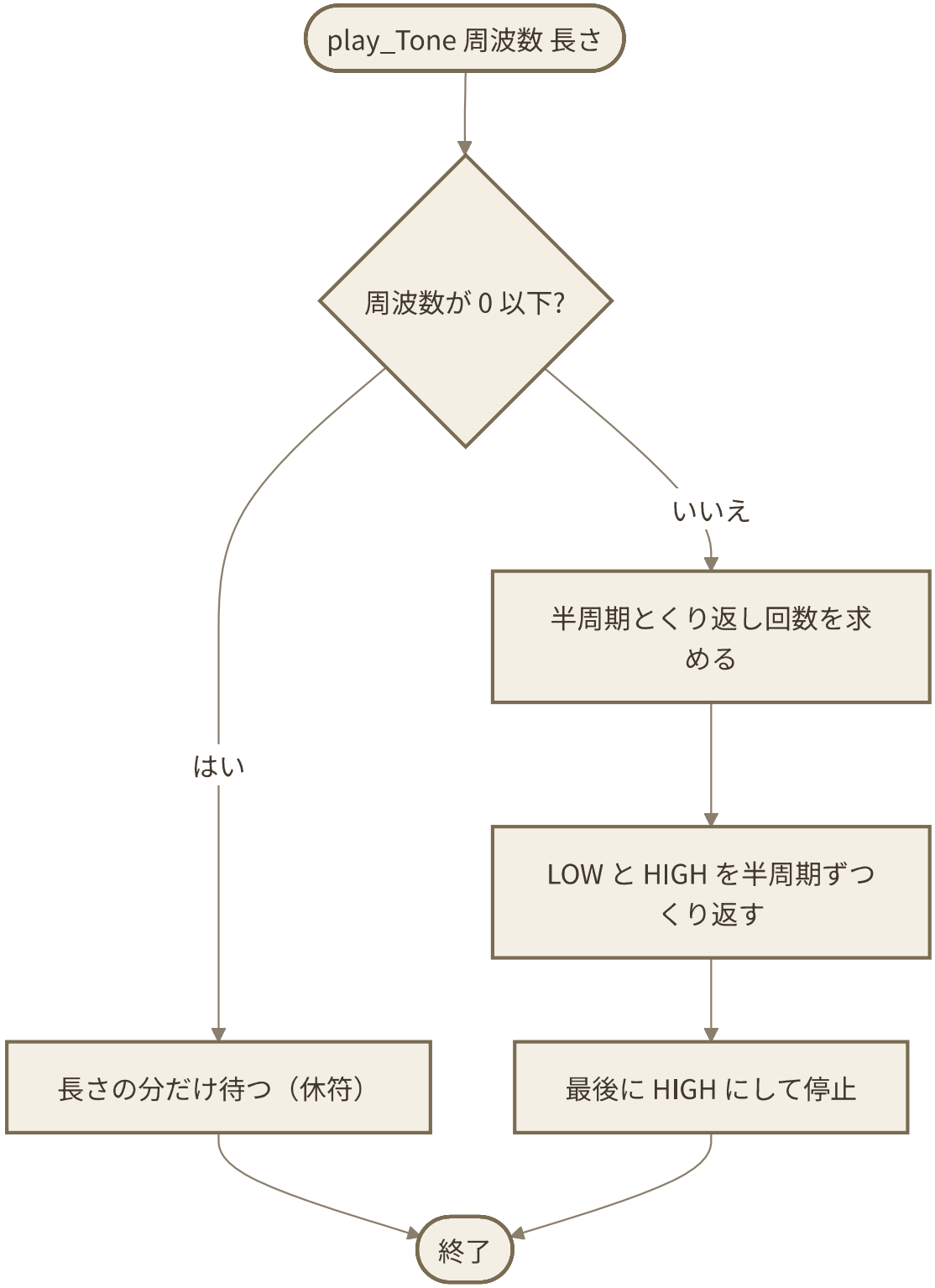


図21　関数 play_Tone

指定した周波数と長さで、1音を鳴らします。



開発環境

開発ツール：Arduino IDE

使用ボード：Arduino Uno

使用言語：C言語（Arduino言語）

+ コード **スケッチ全文（robot_final.ino）**